SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Vulnerability Detection

Course Code:	CSA5803	**Course Title:**	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3
Weightage:	40% of CIA	**Total Marks:**	100
CO3 Statement:	Simulate Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA).		
Student Name:	A Austin	**Reg. No.:**	192421004
Section / Batch:	2024 B Tech IT	**Date:**	13/08/26

Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

Question Type	Focus Area	Questions	Marks
DAST SIMULATION	Application based	Q1	Q1 –100
	GRAND TOTAL:	**100 Marks**	

Code of Conduct

I A Austin (192421004)*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: A Austin

Vulnerability Detection – ASSESSMENT TOOL 3

CSA5803 – DevSecOps Engineering and Application Security

Q1. Vulnerability Detection and Security Verification Simulation

1. Understanding of the Problem

The proposed solution automates security verification within a CI/CD pipeline using Static Application Security Testing (SAST), Dynamic Application Security Testing (DAST), and Software Composition Analysis (SCA). It detects vulnerabilities at source-code, dependency, and runtime levels, classifies their severity, applies remediation, performs re-testing, verifies the fixes, and evaluates security and system performance before and after mitigation. This directly addresses the assessment requirements: vulnerability detection and severity classification, severity analysis using an appropriate metric such as CVSS, a workflow covering detection, remediation, re-testing and verification, and performance evaluation.

2. Objectives

- Detect source-code vulnerabilities using SAST.
- Identify vulnerable or outdated third-party dependencies using SCA.
- Detect runtime application weaknesses using DAST.
- Classify findings by severity and assess impact using CVSS-oriented analysis.
- Automate security gates so unacceptable findings do not progress.
- Re-test fixes and compare security and performance before and after mitigation.

3. CI/CD Security Workflow

Developer Commit → Source Repository → Build & Unit Tests → SAST → SCA → Build Artifact → Test/Staging Deployment → DAST → Security Gate → Remediation → Re-test → Verification → Release.

Control	Target	Purpose	Output
SAST	Source code	Find coding/security weaknesses before runtime	SAST report
SCA	Direct/transitive dependencies	Find known component vulnerabilities	SCA report
DAST	Running application	Find runtime and configuration weaknesses	DAST report
Security Gate	Combined findings	Apply severity/release policy	Pass/Fail decision
Verification	Remediated application	Confirm fixes and detect regressions	Approval evidence

4. Simulated Vulnerability Detection

ID	Tool	Finding	Severity	Impact	Remediation
V-01	SAST	Unsafe database query using user-controlled input	High	Possible unauthorized database operations/data access	Use parameterized queries and server-side validation.
V-02	SAST	Sensitive secret embedded in source/configuration	High	Possible unauthorized access to a protected service	Remove hard-coded secret; use protected secret/environment management.
V-03	SCA	Third-party dependency with known vulnerability	High	Application inherits component-level security risk	Upgrade to supported secure version and re-scan.
V-04	SCA	Outdated/unnecessary dependency	Medium	Increased attack surface and maintenance risk	Update, replace, or remove after compatibility testing.
V-05	DAST	Missing/weak security response headers	Medium	Reduced browser-side security hardening	Configure appropriate security headers.
V-06	DAST	Verbose application error response	Low	May disclose implementation information	Return generic errors and securely log diagnostics.

5. Severity Analysis Using CVSS

CVSS can be used to consistently represent technical vulnerability severity using characteristics such as attack conditions, privileges, user interaction, scope, and impact. The actual numerical score should be taken from the scanner or calculated from the applicable CVSS vector; it should not be fabricated when technical evidence is unavailable.

Finding	Priority	Pipeline Decision	Justification
V-01 Unsafe query	High	Block until remediated	User-controlled input can affect backend database operations.
V-02 Exposed secret	High	Block until remediated	Secret exposure can affect protected resources.
V-03 Vulnerable dependency	High	Block until reviewed/remediated	Known dependency risk should be controlled before release.

V-04 Outdated dependency	Medium	Review and remediate	Risk depends on actual exposure and known issues.
V-05 Weak headers	Medium	Remediate before release	Configuration weakness reduces defensive controls.
V-06 Verbose errors	Low	Track and remediate	Information disclosure has lower direct impact than high-risk findings.

6. Security Verification Workflow

1. Detection – Execute SAST, SCA and DAST at the appropriate CI/CD stages and collect evidence.
2. Classification – Deduplicate findings, identify affected components, classify severity and record evidence.
3. Severity Analysis – Evaluate technical impact using an appropriate metric such as CVSS.
4. Security Gate – Compare findings with defined severity thresholds and stop the release when policy is violated.
5. Remediation – Correct source code, dependencies, configuration, or affected components.
6. Re-testing – Re-run the relevant security scan and confirm the original finding is resolved.
7. Verification – Review evidence, check for regressions, and approve release only when security criteria are satisfied.

7. Security Gate Policy

If an unacceptable high/critical finding is detected by SAST, SCA, or DAST, the pipeline fails and deployment is stopped. Medium findings are reviewed according to project policy, while low findings are tracked for remediation. After fixes, the affected scan is executed again. The pipeline proceeds only when the defined release criteria are satisfied.

8. Performance Evaluation

The same controlled workload should be used before and after mitigation. Relevant measures include response time, throughput, error rate, CPU/memory utilization, and CI/CD execution time.

Metric	**Before Mitigation**	**After Mitigation**	**Expected Observation**
Security findings	Baseline findings	Reduced after fixes/re-scan	Improved security posture.
Response time	Record baseline	Measure under same workload	No unacceptable regression.
Error rate	Record baseline	Measure after fixes	Should remain stable or improve.
CPU/Memory	Record baseline	Measure after fixes	Check security-control overhead.
Pipeline duration	Record baseline	Measure after scan integration	Monitor and optimize scan overhead.

9. Expected Results

- SAST detects source-level weaknesses early.
- SCA detects risks inherited from dependencies.
- DAST detects weaknesses in the running application.
- Severity analysis prioritizes remediation.
- Security gates prevent unacceptable findings from progressing.

- Re-testing verifies remediation.
- Before/after measurements reveal security-related performance effects.

10. Result

The proposed CI/CD pipeline provides an integrated security verification workflow. SAST, SCA, and DAST detect vulnerabilities at complementary layers; severity analysis supports prioritization; the security gate controls release progression; remediation and re-testing verify fixes; and performance evaluation checks for unacceptable impact after mitigation.

11. Conclusion

Integrating SAST, DAST, and SCA into CI/CD establishes continuous security verification rather than treating security as a final-stage activity. The workflow follows detection, severity analysis, remediation, re-testing, verification, and performance evaluation, providing a logical and measurable DevSecOps approach.

12. Rubric Alignment

Criterion	Weight	Level 4 – Exemplary Alignment
Understanding of Problem Context	20%	Covers all stated requirements: detection, severity, workflow, verification and performance evaluation.
Application of Concepts	30%	Correctly connects SAST, DAST, SCA, CVSS-oriented analysis, security gates and re-testing.
Problem-Solving Approach	20%	Uses a logical detection → classify → remediate → re-test → verify → release sequence.
Accuracy of Solution	20%	Provides justified findings, remediation actions, release decisions and measurable evaluation criteria.
Clarity	10%	Uses organized headings, workflow, tables, findings, actions and conclusion.

13. Evidence to Include in Final Submission

- CI/CD workflow diagram.
- SAST scan report and screenshot.
- SCA dependency report and screenshot.
- DAST scan report and screenshot from a controlled test environment.
- Severity/CVSS analysis and remediation record.
- Re-testing evidence showing resolved findings.
- Before/after performance measurements and observations.
- Final security-gate pass/fail evidence.